

JAVASCRIPT

Topics covered

Client side scripting with JavaScript

Variables

Functions

Conditions

loops and repetition

Pop up boxes

Objects

HTML DOM and web,

Browser environments

form validation

Events and Event Listeners

What is JavaScript?

JavaScript (JS) is a high-level, interpreted programming language primarily used for making web pages interactive. It is a core technology of the World Wide Web, alongside HTML and CSS. JavaScript enables dynamic content, animations, form validation, and complex web applications.

Why is JavaScript Used?

1. **Client-Side Interactivity** – JavaScript enhances user experience by adding dynamic elements such as animations, pop-ups, and form validation.
2. **Server-Side Development** – Using Node.js, JavaScript can be used for backend development.
3. **Web and Mobile App Development** – Frameworks like React.js and Angular.js enable JavaScript to build modern applications.
4. **Game Development** – JavaScript, along with WebGL and Canvas API, is used to develop web-based games.
5. **Machine Learning & AI** – Libraries like TensorFlow.js allow JavaScript to implement machine learning models in the browser.

Key Features of JavaScript

1. **Interpreted Language** – No need for compilation; it runs directly in the browser.
2. **Lightweight & Fast** – Executes code quickly within the browser.
3. **Event-Driven Programming** – Supports asynchronous programming for better performance.
4. **Object-Oriented & Prototype-Based** – Uses prototypes instead of traditional classes.
5. **Cross-Browser Compatibility** – Works on all major web browsers.
6. **Dynamic Typing** – Variable types are determined at runtime.

Variables

What are Variables in JavaScript?

A **variable** in JavaScript is a container that stores data values. It allows developers to store, retrieve, and manipulate data throughout a program.

Rules for Declaring Variables in JavaScript

1. **Use `var`, `let`, or `const`** – Variables must be declared using one of these keywords.
2. **Variable Names (Identifiers)**
 - Must begin with a letter (`a-z`, `A-Z`), an underscore (`_`), or a dollar sign (`$`).
 - Cannot start with a number (`0-9`).
 - Can contain letters, numbers, underscores, and dollar signs.
 - Cannot be a reserved JavaScript keyword (e.g., `let`, `const`, `if`, `function`).
3. **Case-Sensitive** – `name` and `Name` are different variables.
4. **Meaningful Names** – Use descriptive names for better readability (`userAge` instead of `x`).
5. **No Spaces** – Variable names cannot contain spaces (`user_name` is valid, `user name` is not).

Types of Variables in JavaScript

JavaScript variables can be categorized based on how they are declared and their scope.

1. Based on Declaration Keywords

Keyword	Scope	Can be Reassigned?	Hoisting Behavior
<code>var</code>	Function Scope	 Yes	Hoisted with <code>undefined</code>
<code>let</code>	Block Scope	 Yes	Hoisted but not initialized
<code>const</code>	Block Scope	 No	Hoisted but not initialized

2. Based on Data Types

JavaScript variables can store different types of values.

1. Primitive Data Types (Immutable)

- **String** → Text values (e.g., "Hello")
- **Number** → Integers & floating points (e.g., 10, 3.14)
- **Boolean** → true or false
- **Undefined** → A variable declared but not assigned a value
- **Null** → A variable explicitly set to "empty"
- **Symbol** → Unique identifiers
- **BigInt** → Large integers (BigInt(1234567890123456789n))

2. Non-Primitive Data Types (Mutable)

- **Object** → Key-value pairs `{}`.
- **Array** → List of values `[]`.
- **Function** → Callable block of code.

Operators in JavaScript

Operators in JavaScript are symbols used to perform operations on variables and values. JavaScript provides various types of operators, categorized as follows:

1. Arithmetic Operators

Used to perform mathematical calculations.

Operator	Description	Example	Output
+	Addition	<code>10 + 5</code>	<code>15</code>
-	Subtraction	<code>10 - 5</code>	<code>5</code>
*	Multiplication	<code>10 * 5</code>	<code>50</code>
/	Division	<code>10 / 2</code>	<code>5</code>
%	Modulus (Remainder)	<code>10 % 3</code>	<code>1</code>
**	Exponentiation (Power)	<code>2 ** 3</code>	<code>8</code>
++	Increment	<code>let x = 5; x++</code>	<code>6</code>
--	Decrement	<code>let x = 5; x--</code>	<code>4</code>

2. Assignment Operators

Used to assign values to variables.

Operator	Example	Equivalent To
<code>=</code>	<code>x = 10</code>	<code>x = 10</code>
<code>+=</code>	<code>x += 5</code>	<code>x = x + 5</code>
<code>-=</code>	<code>x -= 5</code>	<code>x = x - 5</code>
<code>*=</code>	<code>x *= 5</code>	<code>x = x * 5</code>
<code>/=</code>	<code>x /= 5</code>	<code>x = x / 5</code>
<code>%=</code>	<code>x %= 3</code>	<code>x = x % 3</code>
<code>**=</code>	<code>x **= 2</code>	<code>x = x ** 2</code>

3. Comparison (Relational) Operators

Used to compare two values.

Operator	Description	Example	Output
<code>==</code>	Equal to (checks value only)	<code>10 == "10"</code>	<code>true</code>
<code>===</code>	Strictly equal to (checks value & type)	<code>10 === "10"</code>	<code>false</code>
<code>!=</code>	Not equal	<code>10 != 5</code>	<code>true</code>
<code>!==</code>	Strictly not equal	<code>10 !== "10"</code>	<code>true</code>
<code>></code>	Greater than	<code>10 > 5</code>	<code>true</code>
<code><</code>	Less than	<code>10 < 5</code>	<code>false</code>
<code>>=</code>	Greater than or equal to	<code>10 >= 10</code>	<code>true</code>
<code><=</code>	Less than or equal to	<code>5 <= 10</code>	<code>true</code>

4. Logical Operators

Used to perform logical operations (AND, OR, NOT).

Ex:

```
let a = 10, b = 20;
```

```
console.log(a > 5 && b > 15); // true
```

```
console.log(a > 15 || b > 15); // true
```

```
console.log(!(a > 5)); // false
```

5. Bitwise Operators

Used for operations on binary numbers.

Operator	Description	Example	Output
<code>&</code>	Bitwise AND	<code>5 & 1</code>	<code>1</code>
<code> </code>	Bitwise OR	<code>5 1</code>	<code>5</code>
<code>^</code>	Bitwise XOR	<code>5 ^ 1</code>	<code>4</code>
<code>~</code>	Bitwise NOT	<code>~5</code>	<code>-6</code>
<code><<</code>	Left shift	<code>5 << 1</code>	<code>10</code>
<code>>></code>	Right shift	<code>5 >> 1</code>	<code>2</code>
<code>>>></code>	Zero-fill right shift	<code>5 >>> 1</code>	<code>2</code>

6. Ternary (Conditional) Operator

A shorthand for `if-else` statements.

Syntax:

`condition ? value_if_true : value_if_false;`

Ex:

```
let age = 18;
```

```
let status = (age >= 18) ? "Adult" : "Minor";
```

```
console.log(status); // Output: Adult
```

7. Type Operators

Used to check data types.

Operator	Description	Example	Output
<code>typeof</code>	Returns data type	<code>typeof 10</code>	<code>"number"</code>
<code>instanceof</code>	Checks if an object belongs to a class	<code>obj instanceof Array</code>	<code>true</code> or <code>false</code>

8. Nullish Coalescing Operator (??)

Returns the right-hand value if the left-hand value is `null` or `undefined`.

Ex:

```
let value = null;
```

```
let result = value ?? "Default Value";
```

```
console.log(result); // Output: Default Value
```


Types of Mode in Javascript

In JavaScript, you can add code to your HTML in three main ways: **inline**, **internal**, and **external**.

1. Inline Javascript:

Inline JavaScript is written directly within an HTML element, using the `onclick`, `onmouseover`, or other event attributes. It's typically used for small snippets of code or simple interactions.

Example 1

```
<button onclick="alert('Hello!')">Click Me</button>
```

Output : Hello!

Example 2:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>Inline JavaScript Example</title>
```

```
</head>
```

```
<body>
```

```
    <button onclick="alert('This is an inline JavaScript alert!')">Click Me</button>
```

```
</body>
```

```
</html>    //Output: This is an inline JavaScript alert!
```

2. Internal JavaScript

Internal JavaScript is written within a `<script>` tag inside the HTML document, typically in the `<head>` or at the bottom of the `<body>` section.

Example 1:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Internal JavaScript</title>
```

```
  <script>
```

```
    function showMessage() {  
      alert('Hello from Internal JavaScript!');  
    }
```

```
</script>
```

```
</head>
```

```
<body>
```

```
  <button onclick="showMessage()">Click Me</button>
```

```
</body>
```

```
</html> //Output: Hello from Internal JavaScript!
```

3. Internal JavaScript

External JavaScript involves placing the JavaScript code in a separate `.js` file. This file is then linked to the HTML document using the `<script>` tag, usually at the end of the body section.

Example 1:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
  <head> <meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>External JavaScript</title>

<script src="script.js"></script>

</head>

<body>

  <button onclick="showMessage()">Click Me</button>

</body>

</html>  //Output: Hello from External JavaScript!
```

JavaScript File (script.js):

```
function showMessage() {
  alert('Hello from External JavaScript!');
}
```

Inline vs Internal vs External

Type	Location	Use Case	Pros	Cons
Inline JavaScript	Directly in HTML tags	Small snippets of code or simple interactions	Easy to implement, quick to test	Hard to maintain, messy for large projects
Internal JavaScript	Inside <code><script></code> tag in HTML	Small to medium-sized scripts	More organized than inline, still within the HTML	Not ideal for larger projects, file size increases
External JavaScript	Separate <code>.js</code> file	Large projects, shared scripts across multiple pages	Clean, reusable, easy to maintain	Requires extra HTTP request, setup complexity

What is a Function in JavaScript?

A function in JavaScript is a reusable block of code that performs a specific task when invoked. It helps in modularizing code, making it more readable, reusable, and maintainable

Why Use Functions?

Code Reusability – Write once, use multiple times.

Modularity – Divide code into logical sections.

Maintainability – Easier to debug and update.

Avoid Redundancy – Prevent repetition of the same logic.

Improves Readability – Makes code structured and easy to understand.

Types of Functions in JavaScript

1. Function Declaration (Named Function)

A function is defined using the `function` keyword with a name and can be called anywhere in the code due to **hoisting**.

syntax:

```
function functionName(parameters) {  
    // Code to execute  
    return value; // (Optional)  
}
```


Ex:

```
function greet(name) {  
    return "Hello, " + name + "!";  
}  
  
console.log(greet("Adam")); // Output: Hello, Adam!
```

Hoisting: This function can be called before its definition.

2. Function Expression (Anonymous Function)

A function assigned to a variable is called a **function expression**. Unlike function declarations, they are **not hoisted**.

Syntax:

```
const functionName = function(parameters) {  
    // Code  
    return value;  
};
```

Ex:

```
const add = function(a, b) {  
    return a + b;  
};
```

console.log(add(5, 3)); // Output: 8

Not Hoisted: Calling `add(5, 3)` before the definition would throw an error

3. Arrow Functions (ES6)

Introduced in ES6, arrow functions provide a shorter syntax for writing functions

Syntax:

```
const functionName = (parameters) => {  
    // Code  
    return value;  
};
```

Ex:

```
const multiply = (a, b) => a * b;  
console.log(multiply(4, 5)); // Output: 20
```

Implicit Return: If there is only one statement, `{ }` and `return` can be omitted.

Conditional Statements in JavaScript

Conditionals in JavaScript allow you to execute different code based on certain conditions. They control the flow of a program by executing specific blocks of code only when certain conditions are met.

1. **if** Statement

The **if** statement executes a block of code only if the given condition evaluates to **true**.

Syntax:

```
if (condition) {  
    // Code to execute if condition is true  
}
```

Ex:

```
let age = 18;  
  
if (age >= 18) {  
    console.log("You are eligible to vote.");  
}
```

// Output: You are eligible to vote.

If **age** is 18 or more, the message will be printed. Otherwise, nothing happens.

2. **if-else** Statement

The **if-else** statement executes one block of code if the condition is **true**, and another block if the condition is **false**.

Syntax:

```
if (condition) {  
    // Code to execute if condition is true  
}  
else {  
    // Code to execute if condition is false  
}
```

Ex:

```
let age = 16;  
  
if (age >= 18) {  
    console.log("You are eligible to vote.");  
}  
else {  
    console.log("You are not eligible to vote.");  
}
```

// Output: You are not eligible to vote.

3. **if-else if-else** Statement

When there are multiple conditions to check, we use **else if** to specify additional conditions.

Syntax:

```
if (condition1) {  
    // Code for condition1  
} else if (condition2) {  
    // Code for condition2  
} else {  
    // Code if no conditions are met  
}
```

Ex:

```
let marks = 75;
```

```
if (marks >= 90) {
```

```
    console.log("Grade: A");
```

```
} else if (marks >= 75) {
```

```
    console.log("Grade: B");
```

```
} else if (marks >= 50) {
```

```
    console.log("Grade: C");
```

```
} else {
```

```
    console.log("Grade: F");
```

```
}
```

```
// Output: Grade: B
```


4. **switch** Statement

The **switch** statement is used when we have multiple cases to compare with a single variable. It is an alternative to multiple **if-else if** conditions.

Syntax:

```
switch (expression) {  
    case value1:  
        // Code to execute if expression === value1  
        break;  
    case value2:  
        // Code to execute if expression === value2  
        break;  
    default:  
        // Code to execute if none of the cases match  
}
```

Ex:

```
let day = "Monday";
```

```
switch (day) {
```

```
  case "Monday":
```

```
    console.log("Start of the workweek.");
```

```
    break;
```

```
  case "Friday":
```

```
    console.log("Weekend is near!");
```

```
    break;
```

```
  case "Sunday":
```

```
    console.log("Weekend!");
```

```
    break;
```

```
  default:
```

```
    console.log("Just another day.");
```

```
}// Output: Start of the workweek.
```

Loops in JavaScript:

Loops in JavaScript allow us to execute a block of code multiple times. They are useful when we need to perform repetitive tasks, iterate through arrays, process user inputs, or execute logic until a condition is met.

Types of Loops in JavaScript

1. **for** Loop

The **for** loop is used when the number of iterations is known in advance

Syntax:

```
for (initialization; condition; update) {  
    // Code to execute  
}
```

Ex:

```
for (let i = 1; i <= 5; i++) {  
    console.log("Iteration:", i);  
}
```

//output

Iteration: 1

Iteration: 2

Iteration: 3

Iteration: 4

Iteration: 5

2. `while` Loop

The `while` loop is used when the number of iterations is **not known in advance**. The loop runs **as long as the condition is true**.

Syntax:

```
while (condition) {  
    // Code to execute  
}
```

Ex:

```
let i = 1;  
while (i <= 5) {  
    console.log("Iteration:", i);  
    i++; // Increment to avoid infinite loop  
}
```

3. **do-while** Loop

The **do-while** loop is similar to **while**, but it ensures that the loop **executes at least once**, even if the condition is false.

Syntax:

```
do {  
    // Code to execute  
} while (condition);
```

Ex:

```
let i = 1;  
  
do {  
    console.log("Iteration:", i);  
    i++;  
} while (i <= 5);
```

4. **for-in** Loop

The **for-in** loop is used to iterate over the properties of an object.

Syntax:

```
for (let key in object) {  
    // Code to execute  
}
```

Ex:

```
let person = { name: "Adam", age: 20, city: "London" };  
  
for (let key in person) {  
    console.log(key + ":", person[key]);  
}
```

Output:

name: Adam

age: 20

city: London

5. `for-of` Loop

The `for-of` loop is used to iterate over iterable objects like **arrays, strings, Maps, Sets, etc.** It **does not work for objects**.

Syntax:

```
for (let element of iterable) {
```

```
    // Code to execute
```

```
}
```


Example with an Array:

```
let numbers = [10, 20, 30, 40];
```

```
for (let num of numbers) {
```

```
    console.log(num);
```

```
}
```

Output:

10

20

30

40

Example with a String:

```
let word = "Hello";  
for (let char of word) {  
    console.log(char);  
}
```

Output:

H

e

l

l

o

Loop Control Statements

JavaScript provides special statements to control the execution of loops:

1. **break** Statement:

The **break** statement is used to **exit a loop prematurely** when a certain condition is met.

Ex:

```
for (let i = 1; i <= 10; i++) {  
    if (i === 5) {  
        break; // Stops the loop when i is 5  
    }  
    console.log(i);  
}
```

The loop stops when **i** is 5.

2. **continue** Statement:

The **continue** statement **skips** the current iteration and moves to the next one.

Ex:

```
for (let i = 1; i <= 5; i++) {  
    if (i === 3) {  
        continue; // Skips iteration when i is 3  
    }  
    console.log(i);  
}
```

When **i** is 3, the loop **skips** that iteration but continues with 4 and 5.

Objects

Objects in JavaScript are **collections of key-value pairs**. They allow us to store, organize, and manipulate data efficiently. Objects are **one of the most important concepts** in JavaScript because they form the basis of almost everything in the language, including functions, arrays, and even JavaScript itself.

Example of an Object:

```
let person = {  
  name: "Adam",  
  age: 20,  
  city: "London",  
  greet: function() {  
    console.log("Hello, my name is " + this.name);  
  }  
};
```

`name`, `age`, and `city` are **properties** (key-value pairs).

`greet()` is a **method** (a function inside an object).

`this` refers to the **current object** (`person`).

Accessing Object Properties:

You can access object properties in two ways:

1. Using Dot Notation (.)

```
let person = { name: "Adam", age: 20 };
```

```
console.log(person.name); // Adam
```

The above method is Preferred for **known** properties.

2. Using Bracket Notation ([])

Ex:

```
let person = { name: "Adam", age: 20 };
```

```
console.log(person["age"]); // 20
```

Useful when the property name **contains spaces** or **is dynamic**

Modifying Object Properties

1. Updating a Property

Ex:

```
let person = { name: "Adam", age: 20 };
```

```
person.age = 21; // Updating property
```

```
console.log(person.age); // 21
```

2. Adding a New Property

```
person.city = "London";
```

```
console.log(person.city); // London
```

3. Deleting a Property

```
delete person.city;
```

```
console.log(person.city); // undefined
```

Checking Property Existence

1. Using **in** Operator:

```
console.log("name" in person); // true
```

```
console.log("city" in person); // false
```


2. Using `hasOwnProperty()`

```
console.log(person.hasOwnProperty("name")); // true  
console.log(person.hasOwnProperty("city")); // false
```

Iterating Over Object Properties

1. Using `for-in` Loop

```
let person = { name: "Adam", age: 20, city: "London" };  
  
for (let key in person) {  
    console.log(key + ": " + person[key]);  
}
```

Output:

name: Adam

age: 20

city: London

2. Using `Object.keys()`

```
console.log(Object.keys(person)); // ["name", "age", "city"]
```

3. Using `Object.values()`

```
console.log(Object.values(person)); // ["Adam", 20, "London"]
```

4. Using `Object.entries()`

```
console.log(Object.entries(person));
```

```
// [["name", "Adam"], ["age", 20], ["city", "London"]]
```

Popup Boxes:

Popup boxes in JavaScript are used to **interact with users** by displaying messages, taking input, or asking for confirmation. JavaScript provides **three types of popup boxes**:

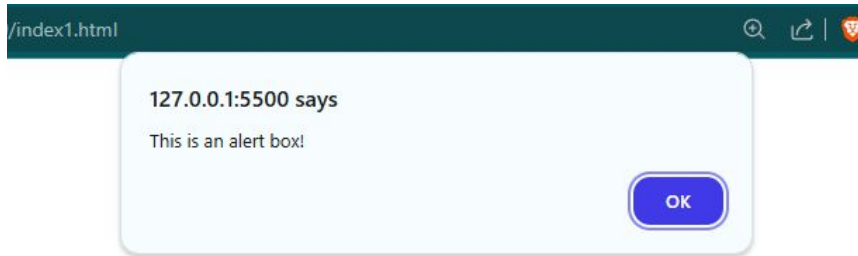
1. Alert Box (**alert()**)

The **alert()** method is used to display a **simple message** to the user. The user can only click "**OK**" to close it.

Ex:

```
alert("This is an alert box!");
```

Output:



✓ The message **pauses execution** until the user clicks "OK".

Use Case:

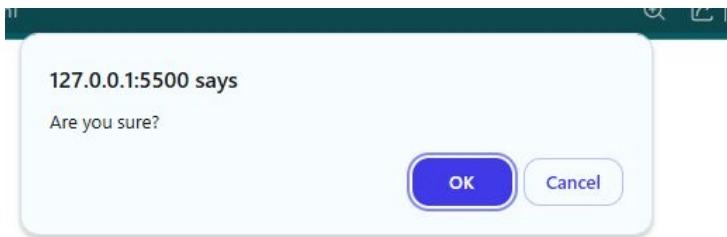
- Notifications
- Error messages
- Warnings

2. Confirm Box (`confirm()`)

Ex:

```
let result = confirm("Are you sure?");
```

Output:



✓ it **returns a boolean** (`true` or `false`), which allows us to make decisions.

Use Case:

- **Asking for confirmation** before performing a critical action.
- **Deleting data** or **submitting a form**.

3. Prompt Box (`prompt()`)

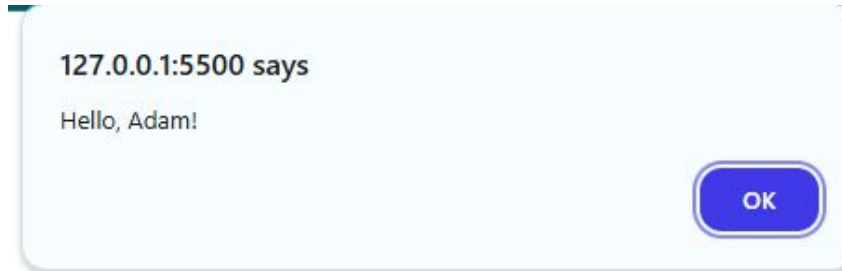
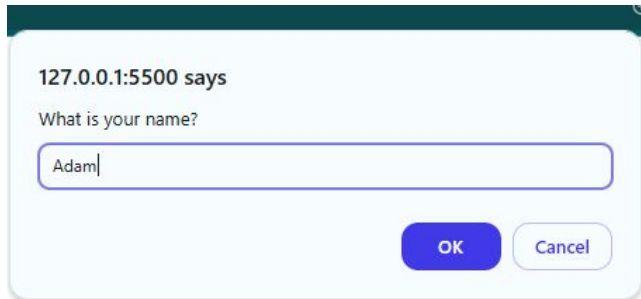
The `prompt()` method is used to **take user input**. It has:

- **An input field**
- **Two buttons: OK & Cancel**
- If the user clicks OK, the entered text is returned.
- If the user clicks Cancel, `null` is returned.

Ex:

```
let name = prompt("What is your name?");  
  
if (name) {  
    alert("Hello, " + name + "!");  
} else {  
    alert("You didn't enter anything.");  
}
```

Output:



✓ If the user **presses OK**, the input is stored in **name**.

✓ If the user **presses Cancel**, **null** is returned.

Use Case:

- **Asking for user details** (name, age, email, etc.).
- **Taking input dynamically** without using an HTML form.

Browser Environments in Detail:

The **browser environment** is the execution environment where JavaScript runs inside a web browser. It consists of various components such as the **Window object**, **Document Object Model (DOM)**, **Browser Object Model (BOM)**, and various APIs that allow interaction with the browser.

Components of the Browser Environment

1. Window Object:

The **window** object is the **global object** in the browser. Every JavaScript function or variable that is declared globally becomes a property of **window**.

Example: Accessing the Window Object

```
console.log(window); // Displays the window object
```

```
console.log(window.innerWidth); // Browser window width
```

```
console.log(window.innerHeight); // Browser window height
```


✓ The `window` object contains **methods and properties** to interact with the browser.

Common Window Methods:

Method	Description
<code>alert("Message")</code>	Displays an alert box
<code>confirm("Are you sure?")</code>	Displays a confirmation box
<code>prompt("Enter your name")</code>	Displays an input box
<code>open("https://google.com")</code>	Opens a new browser window
<code>setTimeout(function, time)</code>	Executes a function after a delay
<code>setInterval(function, time)</code>	Repeats a function at regular intervals

Example: Using `setTimeout`

```
setTimeout(function() {  
    alert("This message appears after 3 seconds");  
}, 3000);
```

✅ The message appears **after 3 seconds**.

2. Document Object Model (DOM)

The DOM is a **tree-like representation** of an HTML document. JavaScript can **modify and manipulate** the DOM dynamically.

Example: Changing the Page Title

```
document.title = "New Page Title";
```

Example: Modifying an Element

```
document.getElementById("heading").innerText = "Updated Heading";
```



The DOM allows JavaScript to modify the web page dynamically.

3. Browser Object Model (BOM)

The BOM allows JavaScript to **interact with the browser itself**, such as:

- Navigating between pages
- Getting browser information
- Handling browser history

3.1. `window.location` (Managing URL)

The `location` object provides information about the **current URL** and allows redirection.

Ex: `console.log(window.location.href);` // Gets the current URL

`window.location.href = "https://google.com";` // Redirects to Google

3.2. **window.history** (Managing Browser History)

The **history** object allows navigation between visited pages.

Ex:

```
window.history.back(); // Go back to the previous page
```

```
window.history.forward(); // Go forward to the next page
```

3.3. **window.navigator** (Getting Browser Info)

The **navigator** object provides details about the browser and device.

Ex:

```
console.log(navigator.userAgent); // Returns the browser's user agent
```

```
console.log(navigator.language); // Returns the user's language
```

3.4. **window.screen** (Getting Screen Dimensions)

The **screen** object provides information about the user's screen.

Ex:

```
console.log(screen.width); // Screen width
```

```
console.log(screen.height); // Screen height
```



The BOM allows JavaScript to interact with the browser without accessing the DOM

HTML DOM

The **HTML DOM (Document Object Model)** is a programming interface for **HTML and XML documents**. It represents the structure of a web page as a **tree-like model** where **each element is an object** that can be manipulated using JavaScript.

1. What is the DOM?

The **Document Object Model (DOM)** is a hierarchical representation of a webpage that JavaScript can interact with.

.Key Features of DOM:

1. Tree Structure:

- The document is represented as a tree (hierarchical structure) where elements are nodes.
- The `<html>` tag is the root node, and all other elements branch from it.

2. Dynamic Interaction:

- Using JavaScript, we can **add, remove, and modify** elements dynamically.

3. Standardized by W3C:

- The DOM is an official standard by the **World Wide Web Consortium (W3C)** and follows a consistent API across browsers.

- It treats the HTML document as a **tree of objects (nodes)**.
- **JavaScript can modify** the content, structure, and style of the web page using the DOM.

1. Structure of the HTML DOM

The DOM represents an HTML document as a **tree of nodes**.

DOM Tree Representation:

```
<html>
```

```
  <head>
```

```
    <title>My Page</title>
```

```
  </head>
```

```
  <body>
```

```
    <h1>Hello</h1>
```

```
    <p>Welcome to my website</p>
```

```
  </body>
```

```
</html>
```

The DOM structure looks like:

Document

└─ html

| └─ head

| | └─ title ("My Page")

| └─ body

| | └─ h1 ("Hello")

| | └─ p ("Welcome to my website")

2. Accessing HTML Elements Using JavaScript

JavaScript provides several ways to select and modify DOM elements

Method	Description
<code>document.getElementById("id")</code>	Selects an element by its ID
<code>document.getElementsByClassName("class")</code>	Selects elements by class name
<code>document.getElementsByTagName("tag")</code>	Selects elements by tag name
<code>document.querySelector("selector")</code>	Selects the first matching element
<code>document.querySelectorAll("selector")</code>	Selects all matching elements

Example: Changing Text

```
document.getElementById("heading").innerText = "New Heading!";
```

Example: Changing Style

```
document.getElementById("heading").style.color = "red";
```

3. Manipulating HTML Elements

Changing Content

Method	Description
<code>element.innerHTML = "New content"</code>	Changes HTML inside an element
<code>element.innerText = "New text"</code>	Changes only the text content
<code>element.textContent = "Text"</code>	Similar to <code>innerText</code> , but includes hidden text

Ex:

```
document.getElementById("heading").innerHTML = "<i>Italicized Text</i>";
```

4. Creating and Removing Elements:

Creating Elements:

```
let newPara = document.createElement("p");  
newPara.innerText = "This is a new paragraph."  
document.body.appendChild(newPara);
```

Removing Elements:

```
let heading = document.getElementById("heading");  
heading.remove();
```

Event Handling:

Event handling in JavaScript refers to the process of detecting and responding to user interactions with a web page, such as **clicks, keypresses, mouse movements, form submissions, and more.**

1. What is an Event?

An **event** is an action that occurs when a user interacts with a webpage. Some common events include:

- **Mouse Events:** `click`, `dblclick`, `mouseover`, `mouseout`, `mousedown`, `mouseup`
- **Keyboard Events:** `keydown`, `keyup`, `keypress`
- **Form Events:** `submit`, `change`, `focus`, `blur`
- **Window Events:** `load`, `resize`, `scroll`

2. How to Handle Events?

There are three main ways to handle events in JavaScript:

1. Using HTML Event Attributes (Inline Events)

You can directly assign an event to an HTML element.

Ex: `<button onclick="alert('Button Clicked!')">Click Me</button>`

Not recommended for large-scale applications as it mixes HTML and JavaScript.

2. Using JavaScript Event Properties (Traditional Method)

You can set event handlers using JavaScript properties.

Ex:

`<button id="myBtn">Click Me</button>`

```
<button id="myBtn">Click Me</button>
```

```
<script>
```

```
  let btn = document.getElementById("myBtn");
```

```
  btn.onclick = function() {
```

```
    alert("Button Clicked!");
```

```
  };
```

```
</script>
```

✅ Limitation: Only one event handler can be assigned per event.

3. Using `addEventListener()` (Recommended Approach)

The best way to handle events is by using `addEventListener()`, which allows multiple event listeners on a single element.

Ex:

```
<button id="myBtn">Click Me</button>
```

```
<script>
```

```
  let btn = document.getElementById("myBtn");
```

```
  btn.addEventListener("click", function() {
```

```
    alert("Button Clicked!");
```

```
  });
```

```
</script>
```



Advantages:

- Allows multiple event listeners.
- Supports event removal (`removeEventListener()`).

3. Event Object (**event**)

When an event occurs, an event object (**event**) is created, which contains details about the event.

Ex:

```
<button id="btn">Click Me</button>
```

```
<script>
```

```
document.getElementById("btn").addEventListener("click", function(event) {
```

```
    console.log(event.type); // "click"
```

```
    console.log(event.target); // The clicked element
```

```
    console.log(event.clientX, event.clientY); // Mouse position
```

```
});
```

```
</script>
```


4. Common Event Types

1. Mouse Events

```
element.addEventListener("click", function() { console.log("Clicked!"); });
```

```
element.addEventListener("dblclick", function() { console.log("Double Clicked!"); });
```

```
element.addEventListener("mouseover", function() { console.log("Mouse Over!"); });
```

```
element.addEventListener("mouseout", function() { console.log("Mouse Out!"); });
```

2. Keyboard Events

Ex: `document.addEventListener("keydown", function(event) {`

```
    console.log("Key Pressed:", event.key);
```

```
});
```

Ex: `document.addEventListener("keyup", function(event) {`

```
    console.log("Key Released:", event.key);
```

```
});
```

5. Event Propagation (Bubbling & Capturing):

When an event occurs inside nested elements, it propagates in two phases:

1. **Bubbling (Default)** – Event goes from the target element up to the root.
2. **Capturing (Trickling)** – Event goes from the root down to the target.

```
<div id="parent">
```

```
  <button id="child">Click Me</button>
```

```
</div>
```

```
<script>
```

```
  document.getElementById("parent").addEventListener("click", function() {
```

```
    console.log("Parent Clicked!");
```

```
  });
```

```
document.getElementById("child").addEventListener("click", function(event) {  
    console.log("Child Clicked!");  
    event.stopPropagation(); // Prevents bubbling  
});  
</script>
```

6. Removing Event Listeners

You can remove an event using `removeEventListener()`.

```
Ex: function myFunction() {  
    console.log("Clicked!");  
}
```

```
let btn = document.getElementById("myBtn");
```

```
btn.addEventListener("click", myFunction);
```

```
// Remove event listener
```

```
btn.removeEventListener("click", myFunction);
```

Note: The function reference must be the same for `addEventListener()` and `removeEventListener()`.

7. Event Delegation:

Instead of attaching events to each child, attach a single event to a parent and use `event.target` to handle specific elements.

Ex:

```
<ul id="list">
```

```
  <li>Item 1</li>
```

```
  <li>Item 2</li>
```

```
  <li>Item 3</li>
```

```
</ul>
```

```
<script>
```

```
  document.getElementById("list").addEventListener("click", function(event) {
```

```
    if (event.target.tagName === "LI") {
```

```
      alert("You clicked: " + event.target.innerText);
```

```
    }
```

```
  });
```

```
</script>
```

✓ Advantages:

- Efficient for handling dynamic content.
- Reduces memory usage.

Form Validation

Form validation ensures that the user enters valid and complete data before submitting a form. JavaScript provides **client-side validation**, which helps improve user experience by catching errors before sending data to the server.

1. Types of Form Validation

1.1 Client-Side Validation (JavaScript & HTML5)

- Runs in the browser before form submission.
- Provides instant feedback.
- Prevents unnecessary server requests.

1.2 Server-Side Validation (PHP, Node.js, etc.)

- Runs on the server after form submission.
 - More secure as it prevents bypassing validation through **developer tools or disabling JavaScript**.
- ♦ **Best Practice:** Use both **client-side** and **server-side** validation for security and a better user experience.

2. Basic HTML5 Form Validation

HTML5 provides built-in validation using attributes like `required`, `pattern`, `maxlength`, `minlength`, etc

Ex:

```
<form id="myForm">
```

```
  <label for="name">Name:</label>
```

```
<input type="text" id="name" name="name" required minlength="3" maxlength="20">
```

```
<label for="email">Email:</label>
```

```
<input type="email" id="email" name="email" required>
```

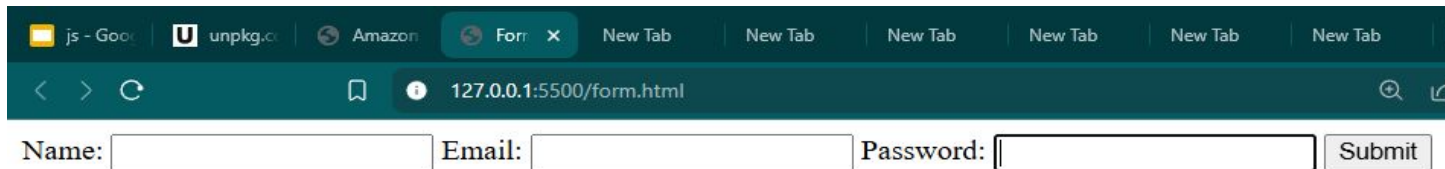
```
<label for="password">Password:</label>
```

```
<input type="password" id="password" name="password" required minlength="6">
```

```
<button type="submit">Submit</button>
```

```
</form>
```


Output:



A screenshot of a web browser window. The address bar shows the URL `127.0.0.1:5500/form.html`. The page content displays a form with three input fields: "Name:", "Email:", and "Password:". Each field is followed by a text input box. To the right of the "Password:" field is a "Submit" button.

✅ **Pros:** Works without JavaScript, faster validation.

❌ **Cons:** Limited customization, no complex logic.

3. JavaScript Form Validation

We use JavaScript to add custom validation logic beyond what HTML5 offers

3.1 Handling Form Submission with JavaScript

```
<form id="myForm">
```

```
  <input type="text" id="name" placeholder="Enter Name">
```

```
<input type="email" id="email" placeholder="Enter Email">
```

```
  <input type="password" id="password" placeholder="Enter Password">
```

```
<button type="submit">Submit</button> </form>
```

```
<script>
```

```
  document.getElementById("myForm").addEventListener("submit", function(event) {  
    event.preventDefault(); // Prevents form submission validateForm();
```

```
  });
```

```
function validateForm() {
```

```
  let name = document.getElementById("name").value;
```

```
let email = document.getElementById("email").value;

let password = document.getElementById("password").value;

if (name.length < 3) {
    alert("Name must be at least 3 characters long.");
    return false;
}

if (!validateEmail(email)) {
    alert("Invalid email format."); return false;
}

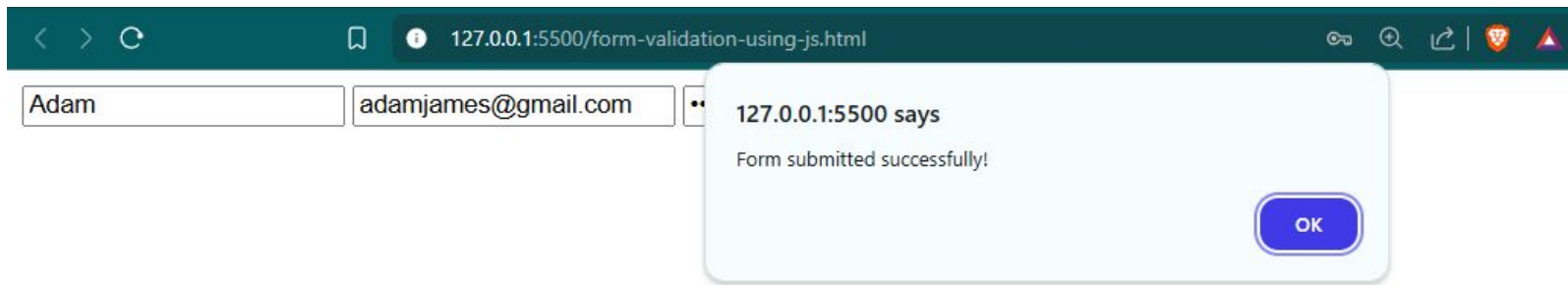
if (password.length < 6) {
    alert("Password must be at least 6 characters long.");
    return false;
}
```

```
alert("Form submitted successfully!");  
  
}  
  
function validateEmail(email) {  
    let regex = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;  
    return regex.test(email);  
}  
  
</script>
```

Explanation for the above code:

The above code validates a form by checking if the name has at least 3 characters, the email follows a valid format, and the password is at least 6 characters long before submission. If any validation fails, an alert message is shown, and submission is prevented; otherwise, a success message is displayed.

Output:



✓ Advantages of JavaScript Validation:

- Customizable logic (e.g., complex password rules, custom messages).
- Instant feedback without reloading the page.

THANK YOU